

Relatório Técnico Consolidado — Massa Hub 3.0

1. VISÃO GERAL E ESCOPO DO PROJETO
2. ARQUITETURA DE DADOS E BACKEND
3. FLUXOS DE TRABALHO E INTEGRAÇÕES (WORKFLOWS)
4. CAPACIDADES DE IA E SISTEMAS AGÊNTICOS
5. INTERFACE DO USUÁRIO (UI/UX)
6. REQUISITOS NÃO-FUNCIONAIS E INFRAESTRUTURA
7. PERFIL DA EQUIPE DE DESENVOLVIMENTO
8. RESUMO EXECUTIVO — PARA AVALIAÇÃO DE PLATAFORMA LOW-CODE/AI-AGENTIC
9. CRONOGRAMA & ROADMAP TÉCNICO
10. MATRIZ DE DECISÕES TÉCNICAS

Relatório Técnico Consolidado — Massa Hub 3.0

Status: Fase 2 aberta (2026-07-28) | **Fase 1:** Concluída em produção

Data do relatório: 2026-08-10

Fundador: Gabriel Quental (solo, não-técnico) | **Executado via:** Claude Code

1. VISÃO GERAL E ESCOPO DO PROJETO

1.1 Objetivo Principal

Massa Hub é uma **rede profissional vertical para a Creator Economy brasileira**, posicionada como “o LinkedIn da Creator Economy” (mas com tom, cultura e estética do mercado criativo, não corporativo).

Diferenciador central: a reputação emerge de **trabalho verificável e bilateral** (assinado pelos dois lados: quem fez + quem contratou), não de cadastros automáticos, scores numéricos ou vitrines bonitas. Referência mental explícita: **GitHub**, não Behance.

Segmentação de públicos conectados: - **Assessor/Gestor de Creator** (porta de entrada; super-nó da rede) - **Creator** (supply side; gratuito permanente) - **Profissionais criativos** (foto, vídeo, design, áudio, jurídico) - **Marcas/Empresas** (demanda, contratação)

1.2 Usuários Finais e Natureza da Plataforma

Portal público + autenticado (híbrido): - **Público (sem login):** visualização de perfis (handle) com reputação verificável, leitura de oportunidades publicadas - **Autenticado:** criação/publicação de oportunidades, reivindicação de perfil, assinatura de provas, exploração da rede

Modelo de uso: - **Interna para assessores/agências:** ferramenta de estruturação e orquestração de oportunidades (ticket estruturado por IA) - **Comercial para clientes:** marketplace bilateral (assessor publica → creator se candida; ambos assinam a prova bilateral)

Foco: comercial, voltado para clientes externos (assessores, agências, creators reais). NÃO é ferramenta estritamente interna.

2. ARQUITETURA DE DADOS E BACKEND

2.1 Modelagem de Dados — Núcleo de Reputação

A unidade atômica da Massa Hub é a **Prova** (registro de trabalho real, assinado pelos dois lados).

Tabela: provas (Fonte: migration 2026-06-18)

```
provas {
  id          TEXT          PRIMARY KEY,      -- id de domínio
  (string)
  tipo        TEXT          NOT NULL          -- campanha |
consultoria | festival | negociacao | outro
  titulo      TEXT          NOT NULL,
  descricao   TEXT          NOT NULL,
  resultado   TEXT,         -- nullable (nem
toda prova tem resultado mensurável)
  criador_id  TEXT          NOT NULL,         -- perfilId de
quem realizou
  contratante_id TEXT      NOT NULL,         -- perfilId de
quem contratou
  participantes TEXT[],     -- outros perfis
envolvidos (grafo)
  data        TIMESTAMPTZ NOT NULL,         -- data de
execução/entrega
  criado_em   TIMESTAMPTZ NOT NULL DEFAULT NOW()
}

-- Invariante: criador_id <> contratante_id (CONSTRAINT)
-- Índices: criador_id (busca por perfil)
```

Tabela: assinaturas (Validação bilateral)

```
assinaturas {
  id          UUID          PRIMARY KEY,
  prova_id    TEXT          NOT NULL REFERENCES provas(id) ON
DELETE CASCADE,
  autor_id    TEXT          NOT NULL,         -- perfilId de
quem assinou
  papel       TEXT          NOT NULL,         -- criador |
contratante
  assinado_em TIMESTAMPTZ NOT NULL DEFAULT NOW()
}

-- Invariante: UNIQUE(prova_id, autor_id) – cada parte assina uma
única vez
-- Uma prova só conta como verificada quando ≥2 assinaturas (mínimo
criador + contratante)
```

Regra crítica (D1): Uma Prova é **verificada** apenas quando assinada por ≥2 partes. Antes disso está em estado em_andamento. Não existe score ou nota — apenas contagem de fatos verificáveis.

Tabela: perfis (Nós da rede)

```
perfis {
  id          TEXT          PRIMARY KEY,      -- id de
domínio
```



```
-- Índices: autor_id (listar oportunidades de um assessor), status,  
criado_em (ordenação)
```

Squad-aware: suporta múltiplos papéis com quantidades. Ex.: “2x creator de beleza + 1x creator de lifestyle” é representado como papeis: [{ funcao: "beleza", qtd: 2 }, { funcao: "lifestyle", qtd: 1 }].

2.2 Banco de Dados

Sistema: PostgreSQL via **Supabase** (projeto isolado, novo)

Versão Supabase: documentada no package.json como
@supabase/supabase-js@2.108.2

Acesso: - **Server-side (Node.js):** createServerClient via SDK
@supabase/ssr - **Persistência:** via service role
(SUPABASE_SERVICE_ROLE_KEY) - **RLS (Row-Level Security):** habilitada,
deny-by-default; sem políticas públicas abertas

Arquitetura hexagonal: Supabase é um **adapter de repositório** na borda da infraestrutura. Domínio não importa Supabase, nunca.

2.3 Migrações e Versionamento

Migrations aplicadas (Postgres): -
20260618000001_oportunidades_e_provas.sql — tabelas base (provas, assinaturas, oportunidades) - 20260618000002_rls_hardening.sql — RLS deny-by-default - 20260722000001_perfis.sql — tabela perfis (nós da rede) - 20260728000001_perfis_email.sql — suporte a notificação por e-mail (Fase 2)

Status: Schema completo aplicado. Banco em produção (~19 perfis, densidade mínima Fase 1).

3. FLUXOS DE TRABALHO E INTEGRAÇÕES (WORKFLOWS)

3.1 Fluxo Central — Oportunidade Estruturada por IA

Fluxo de Publicação (End-to-End):

1. Assessor autenticado acessa /oportunidades/nova
2. Coloca briefing bruto (texto caótico: WhatsApp, e-mail, áudio transcrito)
"Oi! Tô com um job da Natura pra setembro, campanha de skincare, preciso de uns 5 creators do NE, 50k–300k, budget 25 mil, 1 reels + 3 stories + evento em Recife dia 12/09."
3. Clica "Estruturar com IA"
 - ↓ ExtratorDeTicketAnthropic (adapter Anthropic)
 - ↓ System prompt extrai: marca, budget, prazo, local, papéis (squad), entregáveis
 - ↓ Retorna JSON validado + confiança (0.0–1.0)
4. Preview editável do ticket estruturado
 - Realce de origem: trechos do texto bruto sublinhados na cor do campo

- Campo de confiança: se baixa, sinaliza briefing raso
 - Squad-aware: múltiplos papéis com quantidades
5. Assessor revisa + publica (ou edita antes)
 - ↓ Oportunidade nasce em status "aberta"
 - ↓ Notificação para perfis compatíveis (Fase 2: por email)
 6. Candidatura → Prova bilateral
 - Creator se candida ao ticket
 - Assessor confirma (ou indica outra pessoa)
 - Prova é registrada em `provas` table
 - Ambos assinam em `assinaturas` table
 - Prova muda de "em_andamento" para "verificada" (≥2 assinaturas)
 7. Grafo é tecido
 - Participantes da prova criam arestas (quem trabalhou com quem)
 - Lastro é atualizado (contagem de provas verificadas por perfil)
 - Perfil não é mais um nó isolado; é um nó conectado a trabalho real

Aplicação (casos de uso, uma por tarefa): - `PublicarOportunidade` — persist do ticket estruturado - `ExtrairTicket` — orquestra adaptador Anthropic - `CandidatarPerfil` — registra candidatura - `RegistrarProva` — cria prova bilateral - `VincularCreatorAoSquad` — link de assessor → creator → squad da oportunidade - `ReivindicarPerfil` — creator reclama seu perfil (claim profile)

3.2 Fluxo de Reputação — Lastro (Fatos, Não Scores)

Lastro = Agregado de fatos contáveis, derivado de Provas verificadas: - Número de provas verificadas - Número de marcas distintas que assinaram - Número de marcas que voltaram (recorrência) - Número de recomendações de colaboradores

Renderização: - Números crus + rótulo (ex.: “3 marcas • 7 provas • 2 recorrentes”) - Frase-âncora: “Aqui não tem nota nem score. Lastro é trabalho real, assinado por quem participou.” - Sem score de 0-100, sem “Trust Index”, sem “Capital Reputacional”

Ritmo (assinatura visual): - Heatmap de contribuições ao longo do tempo (estilo GitHub contribution graph) - Escala violeta: #F0EFE8 → #DCD8FB → #A99CFF → #6C5BFF → #2B1FA8 - **Só renderizar com volume real.** Heatmap vazio comunica o oposto do pretendido.

3.3 Integrações com Sistemas Externos

Anthropic (Claude API)

- **Modelo:** Claude Haiku 4.5 (claude-haiku-4-5-20251001, trocável via `ANTHROPIC_MODEL` env var)
- **Função:** extração estruturada de briefings em tickets (uma chamada, um agente — D10)
- **Adapter:** `ExtratorDeTicketAnthropic` (arquivo: `src/infrastructure/anthropic/ExtratorDeTicketAnthropic.ts`)
- **System prompt:** iterativo, construído com briefings reais (nunca “finalizado” cedo)
- **Validação:** schema Zod antes de montar preview
- **Telemetria:** log estruturado por extração (confiança, latência, tokens) para decisão de reavaliação (D12)
- **Carga:** ~50 oportunidades/mês = centavos de custo. Critério de reavaliar: quando logs mostrarem confiança caindo ou volume real importar em custo

Resend (Envio de E-mail)

- **Função:** notificações de convite (Fase 2 growth loop)
- **Adapter:** EnviadorDeEmailResend (arquivo: `src/infrastructure/email/EnviadorDeEmailResend.ts`)
- **Comportamento degradado:** sem `RESEND_API_KEY` configurada, devolve `enviado: false` + motivo (não quebra o app). Link manual continua funcionando.
- **Remetente:** `RESEND_API_KEY` (sandbox default) ou `EMAIL_REMETENTE` (domínio próprio)
- **Status Fase 1:** não foi usado em produção (sem usuários reais ainda)

Google OAuth / Supabase Auth

- **Autenticação:** Google via Supabase
- **Fluxo:** login → clique em “Entrar com Google” → callback → sessão → middleware refresh automático
- **Persistência:** `usuario_id` (auth UID) ligado ao Perfil reivindicado
- **Middleware:** `src/middleware.ts` — refresh de sessão em every request

Webhooks, Cron, Event-Driven (Não Implementado Fase 1)

- **Status:** Não existe. Fase 1 é pipeline determinístico (uma IA, uma chamada).
- **Fase 2+:** considerar event-driven para:
 - Notificação de nova oportunidade (quando Resend estiver testado)
 - Matching automático por Lastro (quando houver densidade)
 - Negociação assistida (quando features de negociação nascerem)

3.4 Aprovações e Controle de Qualidade

Fluxo de aprovação humana: 1. Assessor publica ticket (IA propõe, assessor aprova via interface) 2. Campo de confiança da IA sinaliza se briefing está raso → pedir mais info 3. Realce de origem (trechos do texto bruto) torna a extração auditável 4. Preview editável: assessor pode corrigir campo-a-campo antes de publicar

Não existe aprovação automática: IA estrutura, humano revisa e decide.

4. CAPACIDADES DE IA E SISTEMAS AGÊNTICOS

4.1 Arquitetura de IA — Um Agente, Uma Chamada (D10)

Decisão consciente: Multi-agente **NÃO é construído na Fase 1**. Critérios objetivos de reabertura registrados (D10, seção 2 do log):

Reavaliar QUANDO:

1. Volume de oportunidades/mês processar pelo agente único mostra gargalo real
2. Funcionalidades de matching/negociação já existem (hoje: zero)
3. Dataset de eval (``eval/ticket/``) mostra platô de qualidade

O que existe: - **Um agente:** Claude Haiku 4.5 - **Uma chamada:** extração de ticket (briefing bruto → JSON estruturado) - **Validação:** schema Zod + regras de negócio - **Logging:** telemetria estruturada (confiança, latência, tokens) para decisão objetiva

4.2 Capacidades Específicas

Extração de Ticket (ExtratorDeTicketAnthropic)

Input: texto bruto (WhatsApp, e-mail, áudio transcrito)

Output: TicketExtraido (JSON)

```
{
  "marca": "Natura",
  "budget": 25000,
  "prazo": "2026-09-12",
  "local": "Recife",
  "regiao": "Nordeste",
  "entregaveis": ["1 reels", "3 stories", "1 evento"],
  "papeis": [
    {
      "funcao": "Creator de Skincare",
      "qtd": 5,
      "nicho": "skincare",
      "seguidoresMin": 50000,
      "seguidoresMax": 300000
    }
  ],
  "confianca": 0.85,
  "origens": [
    { "campo": "marca", "trecho": "job da Natura" },
    { "campo": "budget", "trecho": "budget 25 mil" }
  ]
}
```

Regras do System Prompt: - Se campo não mencionado → null (nunca inventa) - papeis nunca é vazio (infere "Creator" com qtd=1 se necessário) - confiança (0.0-1.0) reflete riqueza do briefing (campos vazios reduzem) - origens: trechos exatos do texto bruto que originaram cada campo

Validação na aplicação: - Zod schema valida antes de montar preview - Erros de parsing lançam exceção clara - Briefing inválido não publica (assessor vê erro, refaz briefing)

Sem RAG, Sem Tomadas de Decisão Autônomas

- **Não usa:** retrieval de documentos, base de conhecimento, busca de contexto externo
- **Não faz:** decidir aprovação, rejeitar candidatura, fazer matching automático
- **Faz apenas:** estruturação de texto bruto em JSON validado

4.3 Orquestração e Handoff Humano-IA

Fluxo (sempre com humano no loop):

Assessor (humano)
↓ cola briefing bruto
IA (Haiku)

↓ estrutura em JSON
Validação (schema Zod)
↓ se inválido: erro claro para assessor
Assessor (humano)
↓ revisa preview + campo de confiança
↓ edita se necessário
↓ publica (ação explícita)
Ticket publica
↓ notifica perfis compatíveis
Criadores (humanos)
↓ se candidatam
Assessor (humano)
↓ aceita / rejeita / indica outra pessoa

Nenhuma decisão autônoma: humano aprova cada passo.

5. INTERFACE DO USUÁRIO (UI/UX)

5.1 Complexidade da UI

Nível: Moderado a Alto

- **Desktop:** layout de dois painéis (lista + detalhe), estilo GitHub Issues / LinkedIn Recruiter
- **Mobile:** fluxo de duas telas (lista em tela cheia → toque abre detalhe com voltar)
- **Componentes customizáveis:** sim, mas dentro de design system estritamente definido
- **Pixel-perfect:** não é a prioridade. Fidelidade ao design system (tokens, tipografia, espaço) é.

5.2 Stack de Frontend

Framework: Next.js 15.5.19 (React 19.1.1)

Styling: Tailwind CSS 4.1.13 (v4, com @theme blocks para tokens)

Estrutura de componentes: modelo de aplicação/infraestrutura (ports) + React na borda

Rotas (Next.js App Router):

/	– Home (público)
/login	– Google OAuth entry
/[handle]	– Perfil público (GET, sem
auth)	
/(autenticado)/	– Protected routes (require
auth)	
/meu-perfil	– Redireciona para
/(auth)/@handle do logado	
/oportunidades	– Lista de oportunidades
(workspace)	
/oportunidades/[id]	– Detalhe da oportunidade +
ações (candidatar, indicar)	
/oportunidades/nova	– Criar oportunidade (Screen
10: estruturar com IA)	
/provas	– Lista de provas do logado
/provas/nova	– Registrar prova bilateral
/reivindicar/[perfilId]	– Claim profile (setter)
/rede	– Exploração / diretório de

perfis (Fase 2)

5.3 Design System & Identidade Visual

Arquivo fonte: src/app/globals.css (bloco @theme Tailwind v4)

Estética: Editorial-criativa com vocabulário de ferramenta de dev (GitHub-like, não SaaS arredondado)

Paleta de Cores

```
--paper:      #FAFAF6 /* fundo */
--card:       #FFFFFF
--ink:        #16151D /* texto principal */
--ink2:       #5C5A66 /* secundário */
--ink3:       #74727D /* terciário (AA WCAG ok) */
--line:       #E8E6DD
--violet:     #6C5BFF /* marca */
--violet-deep: #2B1FA8
--violet-soft: #EFEFFF
--ember:      #FF5A2D /* destaque quente (raro) */
--ok:         #117A53
```

Heatmap Ritmo (violeta): #F0EFE8 → #DCD8FB → #A99CFF → #6C5BFF → #2B1FA8

Tipografia

- **Archivo** (display, pesos 700–900, stretched ~75%) — títulos, tom duro/editorial
- **Inter** — corpo de texto
- **IBM Plex Mono** — dados, numbers, handles, labels, metadados (vocabulário GitHub)

Forma

- Raio de borda: 8–12px (não pílulas exageradas)
- Bordas: 1px em --line, sombras quase ausentes
- Hierarquia: contraste + tipografia, não sombra pesada
- Densidade informacional: alta (como ferramenta de dev)

5.4 Componentes Críticos (Fase 1)

Screen 10 (Publicação de Oportunidade): - Campo grande de texto bruto (lado esquerdo) - Botão “Estruturar com IA” - Preview editável do ticket (lado direito) - **Realce de origem:** trechos do texto bruto sublinhados na cor do campo (assinatura de confiança) - **Campo de confiança:** score 0.0–1.0 sinaliza riqueza do briefing - **Squad-aware:** múltiplos papéis com quantidades - Ação final: “Publicar ticket”

Página Pública /[handle] (Perfil): - Band de identidade: avatar, nome, handle, tipo (assessor/creator, destaque violeta) - Provas em destaque (conteúdo que vende a pessoa) - Ritmo (heatmap) — só se houver dados reais - Lastro (fatos contáveis): sem score, apenas contagem - Sobre, áreas de atuação, marcas, rede

Workspace de Oportunidades: - Lista de tickets (filtráveis por status, marca, prazo) - Detalhe do ticket (estruturado, squad visível, ações: candidatar, indicar) - Candidaturas já registradas (com nomes e handles dos candidatos)

Exploração da Rede (Fase 2): - Diretório de perfis (busca por handle, nicho, marca) - Filtros por tipo (assessor/creator), região, marca - Visualização de Lastro como critério de matching

5.5 Responsividade

Desktop: 2 painéis (lista + detalhe), estilo GitHub Issues

Mobile ($\leq 390\text{px}$): - Navegação: tab bar inferior (safe-area) - Modais: bottom sheet - Textarea: font-size 16px (evita zoom iOS) - Alvos de toque: $\geq 44\text{px}$ - Filtros horizontais: scroll sem barra visível - Workspaces de 2 painéis → fluxo de 2 telas (sequencial)

Auditorias: overflow horizontal checado a cada mudança (scrollWidth vs clientWidth)

6. REQUISITOS NÃO-FUNCIONAIS E INFRAESTRUTURA

6.1 Deployment & Hosting

Plataforma: Vercel (projeto Next.js isolado)

Banco de Dados: Supabase (projeto isolado, PostgreSQL)

Domínio: massahub.com.br (planejado; ainda em staging)

Ci/CD: - Deploy automático via Vercel (push → main) - Migrations: aplicadas manualmente (não automático no deploy) - Secrets: via Vercel environment variables (não em .env local)

6.2 Escalabilidade e Performance

Constraints conhecidas: - **Fase 1:** ~19 perfis, 0 oportunidades em produção (banco vazio na abertura da Fase 2) - **Fase 2 target:** 20-50 oportunidades reais, 200-500 perfis (concentração em assessores) - **Database:** PostgreSQL Supabase (shared tier adequado para Fase 1/2) - **API IA:** Latência ~2-3s por extração (aceitável; assessor não é tempo real)

Otimizações implementadas: - Índices em provas, oportunidades, perfis (autor, status, handle, usuario_id) - RLS deny-by-default (sem queries abertas) - Busca em lote sem N+1 (ex.: buscarPorIds)

Sem cache implementado Fase 1: perfis públicos (/handle) são GET direto do banco. Cache pode entrar em Fase 2 se latência ficar perceptível.

6.3 Segurança

Autenticação

- Google OAuth via Supabase Auth
- Session management: middleware refresh automático
- Token validation: Supabase SSR SDK

Autorização

- RLS deny-by-default (PostgreSQL nível tabela)
- Service role (SUPABASE_SERVICE_ROLE_KEY) para server-side (Node.js actions)
- Anon key apenas para reads públicos (suportará /[handle] em Fase 2 quando RLS público for adicionado)

Dados Sensíveis

- Briefings brutos (contêm marca, budget real, nomes de criadores) — armazenados em `origem_texto_bruto`
- **Medida:** dados sensíveis não vazam em DTO público (DTOs não vazam `usuario_id`, apenas `perfilId`)

Validação

- Input: Zod schemas (extração de ticket, forms de oportunidade, prova)
- Output: DTOs que não expõem internals

Conformidade

- **LGPD:** aplicável (Brasil). Usuários podem solicitar exportação/exclusão (não implementado Fase 1, roadmap Fase 2).
- **GDPR:** não aplicável (público-alvo é Brasil).

6.4 Observabilidade & Logging

Telemetria estruturada: - Extrações de ticket: evento: `"extracao_ticket"`, modelo, confiança, latência (ms), tokens (entrada/saída) - Greppável nos runtime logs do Vercel (para decisão objetiva D12, reavaliação de provedor/modelo)

Não implementado Fase 1: - Dashboards de métrica - Alertas automáticos - Trace distribuída

6.5 Continuidade & Disaster Recovery

Backups: Supabase gerencia (padrão SLA do plano)

RTO/RPO: Não especificado. Fase 1 é low-criticality (base contém ~19 perfis).

Migrations: versionadas em `supabase/migrations/`, aplicáveis manualmente. Reversibilidade testada em staging.

7. PERFIL DA EQUIPE DE DESENVOLVIMENTO

7.1 Estrutura Organizacional

Equipe Executora: - **Fundador (Gabriel Quental):** estratégia, decisões de produto, design, teste de mercado - **Claude Code (IA):** implementação, arquitetura, code review, roadmapping técnico

Não há equipe humana. Construção 100% bootstrapada, solo.

7.2 Perfil de Desenvolvedores Necessários

Para continuar a Massa Hub: - Full-stack (TypeScript/Node.js, React, Next.js) - **Backend especializado:** banco de dados (PostgreSQL), arquitetura hexagonal, testes de domínio - **Frontend:** React, Tailwind, responsividade, design system - **DevOps mínimo:** Vercel, Supabase, secrets/env vars

Não é citizen developer / low-code: o projeto requer código puro. Arquitetura hexagonal, testes isolados, domínio sem sidee-effects — tudo precisa ser explícito.

7.3 Stack de Desenvolvimento

Linguagem & Runtime

- **TypeScript** 5.9.2 (strict mode)
- **Node.js** (Vercel, Next.js runtime)

Framework Principal

- **Next.js** 15.5.19 (App Router, React Server Components)
- **React** 19.1.1

Banco de Dados & ORMs

- **PostgreSQL** (Supabase)
- **Supabase SDK:** @supabase/supabase-js@2.108.2
- **Sem ORM:** queries diretas via SDK (simples, controlado, testável)

Styling

- **Tailwind CSS** 4.1.13 (v4, com @theme)
- **Sem CSS-in-JS:** escrever direto em globals.css

Validação & Type Safety

- **Zod** 3.25.76 (schema validation)
- **TypeScript types** gerados manualmente (sem geração automática de bancos)

Testes

- **Vitest** 3.2.6 (test runner, unit + integration)
- **Modelos de teste:**
 - Domínio: sem banco (mocks das portas)
 - Aplicação: sem IA/banco (mocks dos adapters)
 - Infraestrutura: testes de adapter (se necessário)
- **Coverage target:** domínio ≥90%, aplicação ≥70%

IA / Observabilidade

- **Anthropic SDK:** @anthropic-ai/sdk@0.104.2 (Claude Haiku)
- **Resend SDK:** fetch HTTP direto (sem SDK, 1 dep a menos)

DevOps & Build

- **Vercel** — hosting (Next.js deployment, env secrets, preview deployments)
- **Git** — versionamento (branch claude/projeto-relatorio-tecnico-

s38uei para esta sessão)

Scripts de Desenvolvimento

```
{
  "dev": "next dev",           // dev server local
  "build": "next build",      // build de produção
  "start": "next start",      // run produção local
  "lint": "next lint",        // ESLint
  "test": "vitest run",       // testes uma vez
  "test:watch": "vitest",     // watch mode
  "test:dominio": "vitest run tests/domain", // só domínio
  "eval:ticket": "tsx eval/ticket/run.ts",    // eval de extrações
  // (dataset de test real)
  "context": "repomix --compress ..."      // exporta contexto (para
  // discussões)
}
```

7.4 Padrões de Código & Arquitetura

Hexagonal (Ports & Adapters)

```
src/
├── domain/                -- núcleo, sem deps externos
│   ├── oportunidade/     -- Oportunidade, Squad, Papel
│   ├── prova/            -- Prova, Assinatura, Lastro
│   ├── perfil/           -- Perfil, Handle, Email
│   └── ports/            -- interfaces (PerfilRepositorio,
OportunidadeRepositorio, etc.)
├── application/          -- casos de uso (orquestração via
injeção de portas)
│   ├── PublicarOportunidade.ts
│   ├── ExtrairTicket.ts
│   ├── ReivindicarPerfil.ts
│   └── ... (17 casos de uso Fase 1)
├── infrastructure/       -- adapters (aqui moram deps externas)
│   ├── supabase/         -- repositórios concretos
│   ├── anthropic/        -- ExtratorDeTicketAnthropic
│   └── email/            -- EnviadorDeEmailResend
├── app/                  -- Next.js routes + UI
└── components/           -- React components (UI)
```

Regra crítica: domínio não importa de infrastructure ou app.

Acoplamento unidirecional: app → application → domain; application → ports; infrastructure → ports (implements).

Testes

Domínio: - Entidade: criar, validar invariantes, simular operações - Value Object: criar, validar, comparação - Porta: apenas tipo (interface), sem implementação

```
// tests/domain/perfil/Perfil.test.ts
describe("Perfil", () => {
  it("rejeita perfil pendente sem origem de oportunidade", () => {
    expect(() => Perfil.criar({ tipo: "creator", nome: "Foo",
estado: "pendente" }))
      .toThrow("D7: pendente exige origem");
  });
});
```

Aplicação: - Caso de uso com mocks de porta - Simula fluxo (assessor publica → IA estrutura → assessor aceita → prova registrada)

```
// tests/application/PublicarOportunidade.test.ts
it("publica oportunidade com ticket estruturado", async () => {
  const mockRepositorio = mock(OportunidadeRepositorio);
  const mockExtrator = mock(ExtratorDeTicket);

  mockExtrator.extrair.mockResolvedValue(ticketEsperado);

  const caso = new PublicarOportunidade(mockRepositorio,
mockExtrator);
  const resultado = await caso.executar({ textoBruto: "..." });

  expect(mockRepositorio.salvar).toHaveBeenCalledWith(oportunidadeEsperada);
});
```

Status Fase 1: 109 testes, rodando em CI.

7.5 Processos de Entrega

Branching: - main — produção - develop — integração (não usado, push direto a main) - Feature branches: nomeadas por especialidade (ex: feat/claim-profile) - Esta sessão: claude/projeto-relatorio-tecnico-s38uei

Commits: pequenos, descritivos, em PT-BR

PRs: draft até code review + testes passarem

Merging: squash + merge para main

Deploy: - Automático via Vercel (push main) - Migrations: manual pré-deploy (não automático)

Horário de produção: estável (usuários de teste na base desde 2026-06-18)

8. RESUMO EXECUTIVO — PARA AVALIAÇÃO DE PLATAFORMA LOW-CODE/AI-AGENTIC

8.1 Análise para Plataformas Open-Source

Pergunta crítica: pode um plataforma Low-Code/AI-Agentic Open-Source (ex: Flowise, LangChain, CrewAI) substituir o stack atual?

Resposta técnica: PARCIALMENTE

Pode substituir: - Orquestração de IA (extração de ticket) — sim, Flowise/LangChain conseguem - UI de workflow visual — sim, mas a Massa Hub precisa de frontend customizado

NÃO pode substituir: - **Domínio de negócio:** lógica de Lastro, invariantes D7, regras de Prova bilateral — isso é código puro, não visual - **Persistência hexagonal:** o banco é um adapter, não o núcleo. Plataforma low-code typical trata banco como rainha (UI-driven), não

como servant (domain-driven) - **Teste de domínio isolado:** plataformas low-code rodam sobre framework genérico, testabilidade é secundária - **Deploy lean:** Vercel + Supabase é ~\$0 até escala grande; plataforma hosted é custo fixo

8.2 Trade-offs

Aspecto	Stack Atual (Next.js + Supabase)	Plataforma Low-Code/AI-Agentic
Controle arquitetural	Total (hexagonal, testes, design)	Restrito (forma genérica da plataforma)
Curva de aprendizado	Alta (3+ meses full-stack)	Média (UI visual, mas YAML/config pesado)
Custo inicial	Baixo (~\$0, bootstrap)	Médio-Alto (plano hosted, workflow units)
Customização	Ilimitada (código puro)	Limitada (dentro da plataforma)
Escalabilidade	Controlada (indexação, RLS)	Automática (mas pode ser cara)
Time required	Full-stack (ou solo com IA)	Citizen developer + arquiteto backend
Time-to-value Fase 1	6-8 semanas (pronto em produção)	8-12 semanas (integração, testes)

8.3 Recomendação

Manter stack atual (Next.js + Supabase) porque:

- 1. **Hexagonal já está em lugar:** domínio independente, testável, defensável
- 2. **Custo de migração >> benefício:** reescrever domínio em plataforma genérica é perder o ativo
- 3. **Escala alvo (Fase 2):** 200-500 perfis, ~50 oportunidades/mês — confortável no Vercel/Supabase
- 4. **Critério D10 (IA) não reatualizado:** multi-agente continua fora de escopo; uma chamada Haiku resolve

Se projeto escalasse para: - **Milhares de oportunidades/dia** → revisar Flowise para parallelização - **Equipe de 5+ devs** → considerar LangChain py para shared workflow library - **Restrição de vendor lock-in** → self-hosted plataforma low-code (ex: RunWayML)

Até lá, código puro + teste de domínio isolado > plataforma visual.

9. CRONOGRAMA & ROADMAP TÉCNICO

9.1 Fase 1 (Encerrada — 2026-07-28)

✓ Concluída. Em produção (~19 perfis, ~0 oportunidades no banco, fluxo não foi executado).

Construído: - Módulo de oportunidades (extração IA, preview, publicação) - Sistema de Lastro (provas bilaterais, assinaturas) - Claim profile (assessor + creator) - Página pública /[handle] - Rede tecida

(grafo de relacionamentos) - 109 testes, schema aplicado

9.2 Fase 2 (Aberta — 2026-07-28 até ~2026-09)

Objetivo: rede deixa de ser single-player, suporta pessoas reais entrando, se encontrando, acumulando Lastro.

Escopo liberado (planejado, não iniciado): - Ativação por e-mail (Resend) - Onboarding de quem chega sozinho - Exploração / diretório de perfis - Camada social mínima (conexões, atividade) - Matching por Lastro (vs. score) - Mapeamento assistido de creators (com D7 continuando válida)

Deadline referência: YOUPIX Summit 2026 (2026-09-29)

9.3 Fase 3+ (Quarentena)

Ø Fora de escopo: - Curadoria educacional e clube de ferramentas - Identity OS (pipeline multi-agente) - Sistema multi-agente (orchestrator + agentes autônomos) - Escopo fora da economia criativa

10. MATRIZ DE DECISÕES TÉCNICAS

#	Decisão	Áreas Afetadas	Status	Critério de Reabertura
D1	Reputação = Lastro (fatos), nunca score	Domínio, UI, DB	✓ Decidida	Score é vazio; apenas evidência
D2	Prova bilateral é unidade atômica	Domínio, DB schema	✓ Decidida	Bilateral = difícil forjar
D7	Claim profile ancorado em trabalho real	Domínio, app, UI flow	✓ Decidida	Evita base degradável de perfis fantasma
D10	IA = um agente, uma chamada (não multi-agente)	App, infra, roadmap	✓ Decidida com critério	Volume real + features que orquestrem
D11	Supabase como adapter de repositório	Arquitetura, infra	✓ Decidida	Hexagonal honesta desde dia 1
D12	Provedor IA = Anthropic Haiku (trocável via env)	Infra, custo, roadmap	✓ Decidida com critério	Logs mostram confiança caindo; custo > centavos

Relatório consolidado em: 2026-08-10

Fonte: CLAUDE.md, specs 01-08, migrations, código fonte (src/),
decisoões/log.md

Nível de detalhe: Engenheiro Sênior / Arquiteto de Soluções

Uso proposto: Avaliação de plataforma Open-Source Low-Code/AI-
Agentic como alternative strategy